Credit Card Fraud Detection Using 1D CNN

Author: Thanishk Palaparthy

Course: Deep Learning — Spring 2026

1. Project Goal

The primary objective of this project is to predict fraudulent credit card transactions to enable early detection and prevent financial loss. The central challenge is overcoming extreme class imbalance, as fraudulent transactions constitute only 0.172% of the dataset. This goal was successfully achieved; the proposed 1D Convolutional Neural Network (CNN) demonstrated robust detection capabilities, ultimately achieving an F1-score of 0.7930.

2. Input and Ground Truth

The model takes 30 numerical features as input. Features V1 through V28 are the result of a PCA transformation applied to the original data for confidentiality. The remaining two features are "Time" (seconds elapsed from the first transaction) and "Amount" (transaction value).
The ground truth target is a binary classification label (Class), where 1 indicates a fraudulent transaction and 0 indicates a legitimate transaction.

Changes from Proposal: The final input representation remains consistent with the original proposal; however, our imbalance handling strategy evolved to include testing both class weighting and SMOTE to determine optimal feature extraction.

3. Dataset Description

The Kaggle Credit Card Fraud Detection dataset contains 284,807 credit card transactions made by European cardholders over a two-day period in September 2013. The class distribution is highly imbalanced, comprising 492 fraudulent transactions and 284,315 legitimate transactions. The data is provided in a comma-delimited CSV format.

4. Data Processing

- **Feature Scaling:** StandardScaler was applied to all 30 features to normalize them to zero mean and unit variance.
- **Data Splits:** The dataset was divided into 70% training (199,364 samples) and 30% testing (85,443 samples) using stratified sampling. To monitor training stability and prevent overfitting, 20% of the training partition was isolated as a validation split during model fitting.

- **Imbalance Handling:** The Synthetic Minority Over-sampling Technique (SMOTE) with k_neighbors = 5 was applied only to the training set to prevent data leakage.

5. Methodology

5.1 Baseline Model

A Logistic Regression classifier served as the linear baseline configured with the L-BFGS solver, max_iter = 1000, and class_weight = 'balanced'.

5.2 CNN Model Architecture

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 30, 32)	128
batch_normalization (BatchNormalization)	(None, 30, 32)	128
max_pooling1d (MaxPooling1D)	(None, 15, 32)	0
dropout (Dropout)	(None, 15, 32)	0
conv1d_1 (Conv1D)	(None, 15, 64)	6,208
batch_normalization_1 (BatchNormalization)	(None, 15, 64)	256
max_pooling1d_1 (MaxPooling1D)	(None, 7, 64)	0
dropout_1 (Dropout)	(None, 7, 64)	0
conv1d_2 (Conv1D)	(None, 7, 128)	24,704
batch_normalization_2 (BatchNormalization)	(None, 7, 128)	512
max_pooling1d_2 (MaxPooling1D)	(None, 3, 128)	0
dropout_2 (Dropout)	(None, 3, 128)	0
flatten (Flatten)	(None, 384)	0
dense (Dense)	(None, 256)	98,560
batch_normalization_3 (BatchNormalization)	(None, 256)	1,024
dropout_3 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32,896
batch_normalization_4 (BatchNormalization)	(None, 128)	512
dropout_4 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 1)	129

The proposed architecture is a one-dimensional Convolutional Neural Network (1D CNN). This design was specifically chosen because Conv1D layers treat the 30 input features as a sequence, allowing the network to capture local feature interactions without imposing the artificial two-dimensional spatial structure required by standard 2D CNNs.

Key Design Choices:

- **Filter Progression:** The model utilizes three convolutional blocks with a doubling filter progression (32, 64, and 128 filters). This allows the network to learn increasingly complex, hierarchical representations of the tabular data.
- **Kernel Size and Pooling:** A consistent kernel size of 3 with 'same' padding was selected to safely extract local patterns across the 30 features, followed by MaxPooling1D (pool size = 2) to progressively downsample the spatial dimension and reduce computational load.
- **Regularization Strategy:** To prevent the model from overfitting on the highly imbalanced data, Batch Normalization is applied after every convolutional layer and every dense layer to prevent internal covariate shift. Additionally, an asymmetric dropout strategy was employed: the convolutional blocks use a moderate dropout rate of 0.3, while the parameter-dense fully connected classification head utilizes a heavier dropout rate of 0.4 to aggressively penalize over-reliance on specific flattened features.

5.3 Training Details and Reproducibility

Settings: Adam optimizer (learning rate = 0.001), batch size of 32, maximum 50 epochs. For the foundation model comparison (Section 8), a larger batch size of 1,024 and 15 epochs were used to accommodate the expanded image-based input.

Training stability was smooth; loss curves demonstrated proper convergence behavior without erratic gradient spikes, verifying our network initialization and learning rate scheduling (ReduceLROnPlateau with factor=0.5 and patience=3).

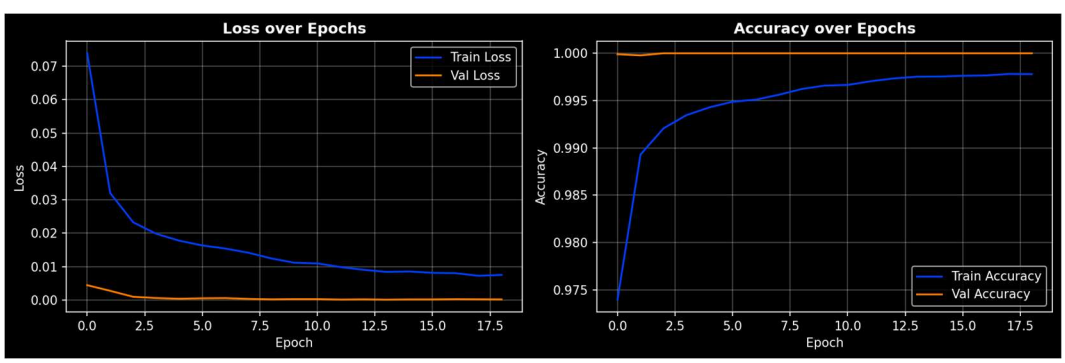

Figure 1: Training Loss and Accuracy History

Comparison with Logistic Regression:

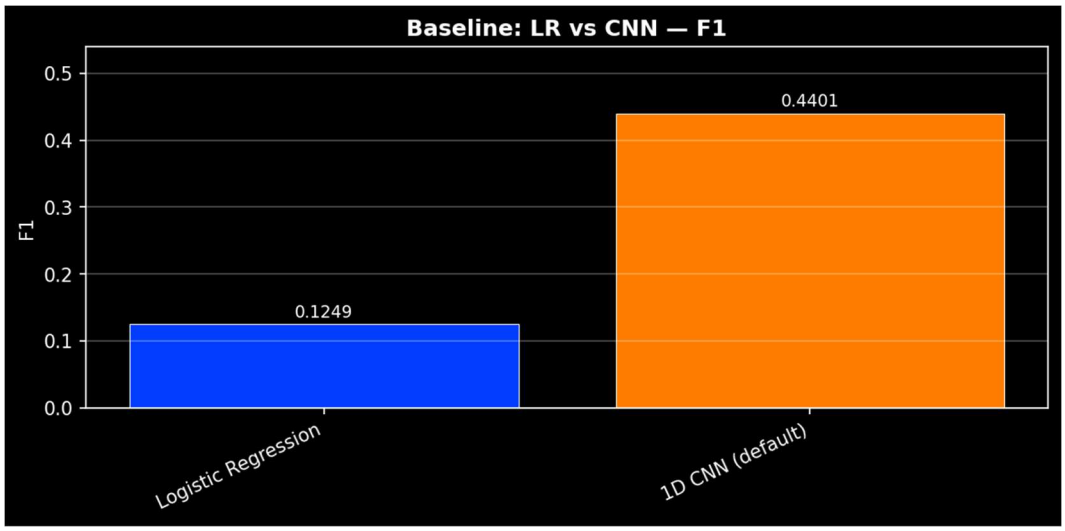

Figure 2: Baseline Comparison F1

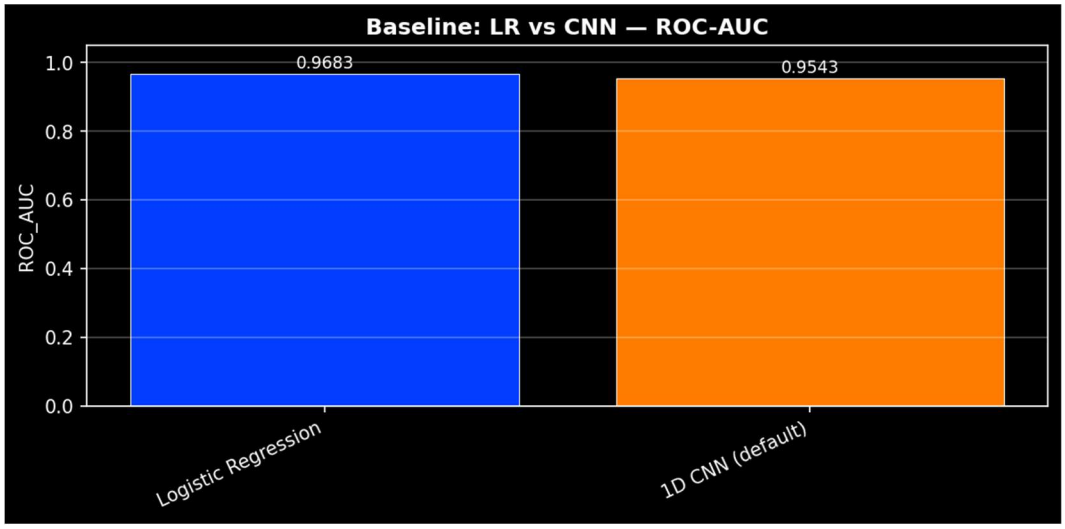

Figure 3: Baseline Comparison ROC

6. Performance Improvement Techniques (Ablation Study)

An ablation study (using a uniform 30 epochs) evaluated three key techniques:

1. **SMOTE Oversampling:** Counterintuitively, the model trained without SMOTE achieved the highest F1-score (0.7930).

Figure 4: Ablation Smote F1

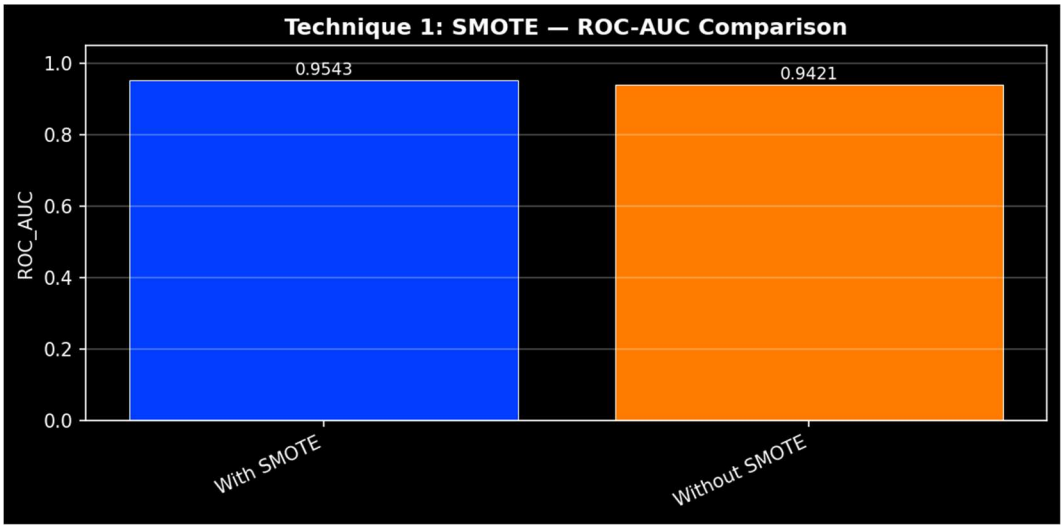

Figure 5: Ablation Smote ROC

2. **Dropout Rate:** Removing dropout entirely (0.0) yielded a strong F1-score of 0.7557, while aggressive dropout (0.5) caused severe underfitting.

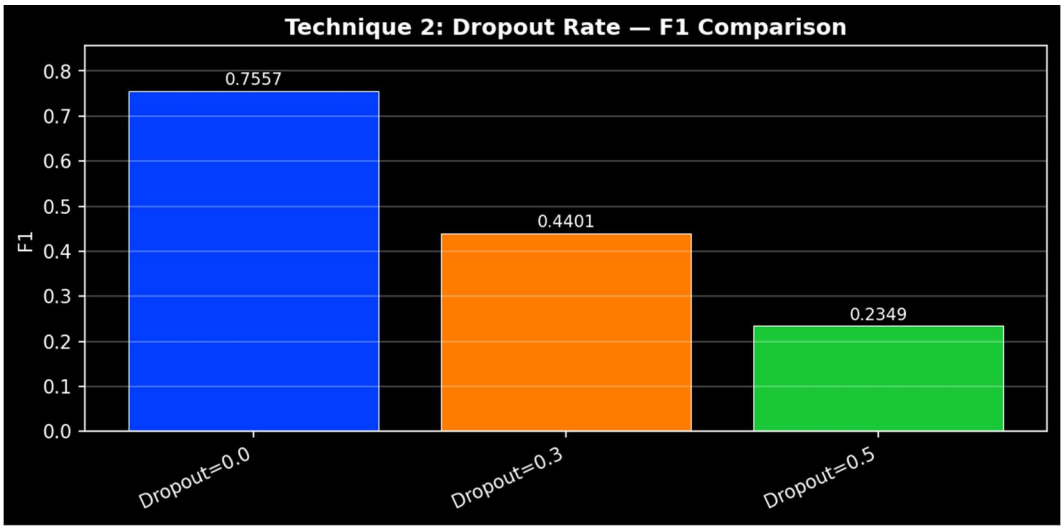

Figure 6: Ablation Dropout F1

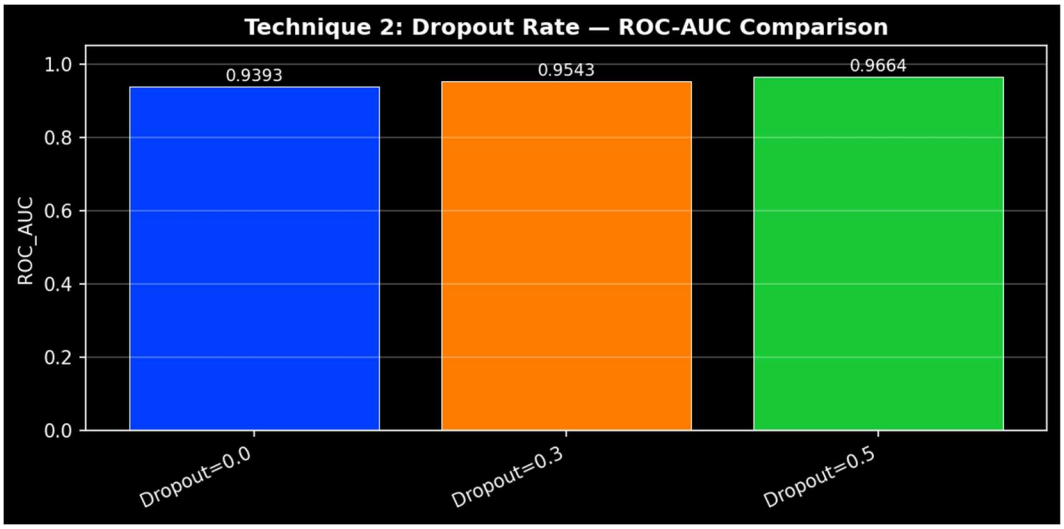

Figure 7: Ablation Dropout ROC

3. **Batch Normalization:** Removing BatchNorm proved catastrophic, dropping the F1-score to 0.2153.

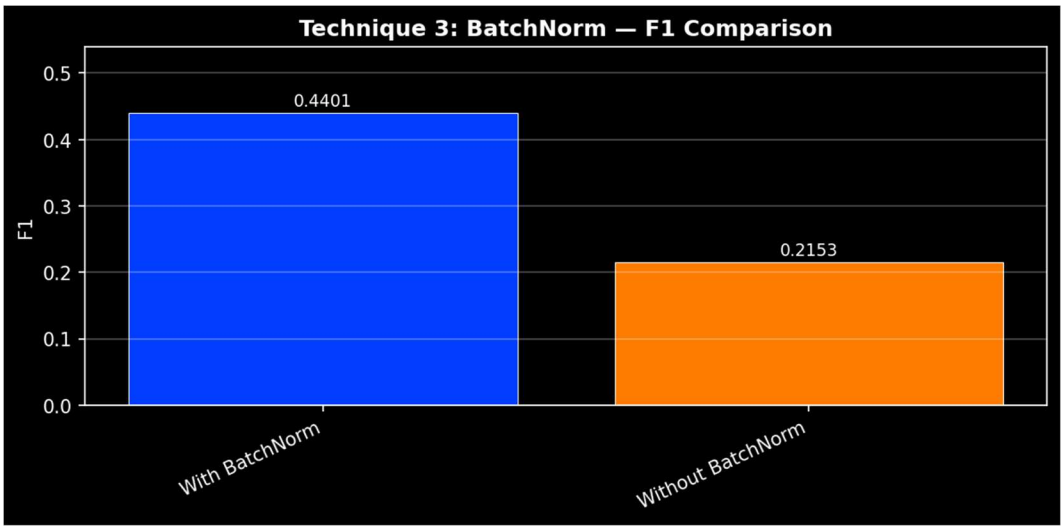

Figure 8: Ablation Batchnorm F1

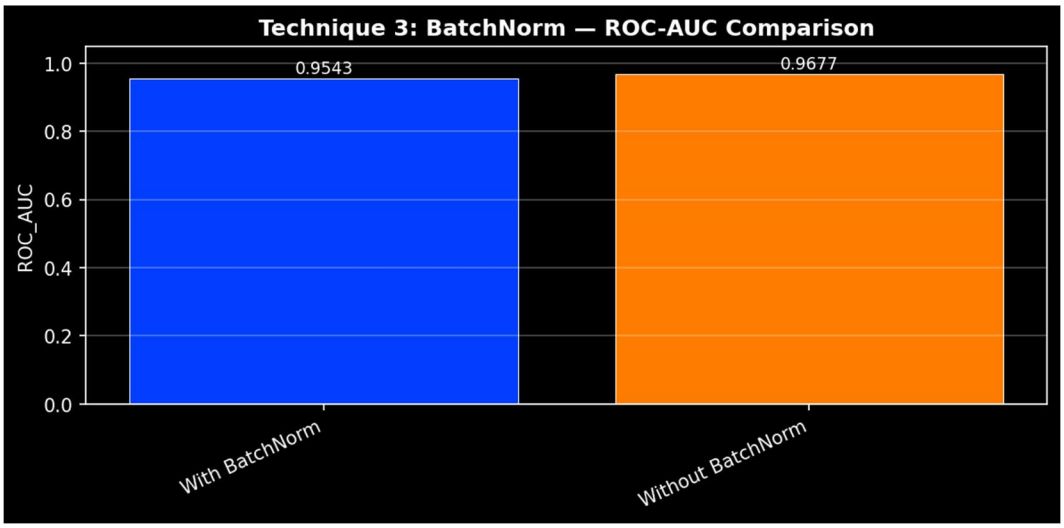

Figure 9: Ablation Batchnorm ROC

7. Performance Evaluation & Comparison of Approaches

Models were evaluated using Precision, Recall, F1-score, ROC-AUC, and PR-AUC, prioritizing PR-AUC and F1 due to the asymmetric costs of misclassification.

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.0673	0.8784	0.1249	0.9683	0.7406
1D CNN (Default w/ SMOTE)	0.2993	0.8311	0.4401	0.9543	0.8052
1D CNN (Best Ablation: No SMOTE)	0.8248	0.7635	0.7930	0.9421	0.8026
MobileNetV2 (Foundation)	0.0148	0.6824	0.0291	0.8758	0.3016

Analysis: While Logistic Regression achieved the highest ROC-AUC (0.9683), its precision completely collapsed (0.0673) at the 0.5 threshold, meaning 93% of its fraud flags were false alarms. The 1D CNN without SMOTE emerged as the best approach.

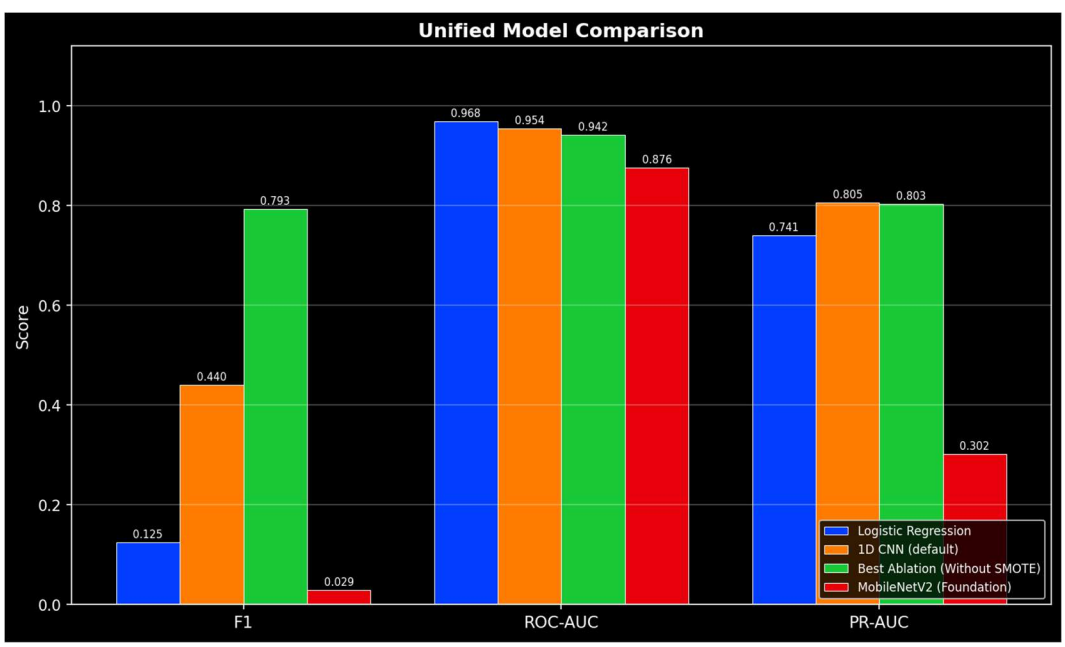

Figure 10: Unified Model Metric Comparison

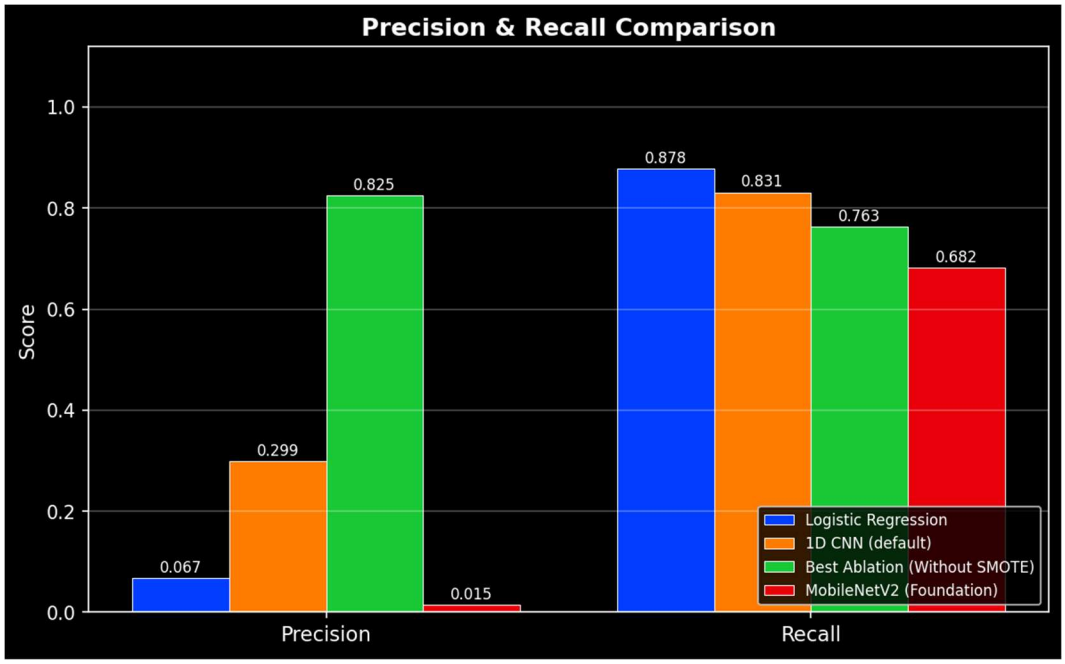

Figure 11: Unified Precision-Recall Curve

8. Foundation Model Comparison

MobileNetV2 (0.4M parameters, ImageNet pre-trained) was evaluated to test transfer learning applicability. The 1D CNN drastically outperformed MobileNetV2 (F1: 0.4333 vs. 0.0291). Note: the CNN F1 here (0.4333) differs slightly from the ablation table value (0.4401) because the foundation comparison script re-evaluates the saved checkpoint independently. This failure is attributed to severe domain mismatch: natural image filters are ineffective for tabular financial data.

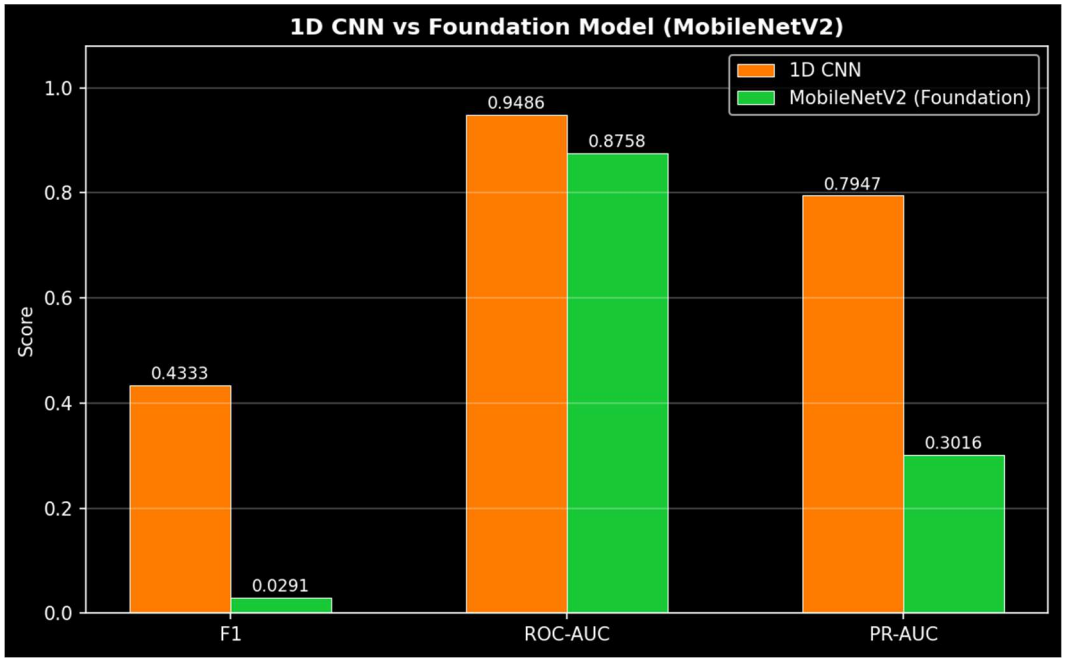

Figure 12: 1D CNN vs. Foundation Model (MobileNetV2)

9. Error and Failure Analysis

An analysis of the default 1D CNN test set errors revealed 23 false negatives (missed fraud) and 304 false positives (false alarms).

- **High-Confidence Errors:** False negatives had a mean predicted probability of just 0.0005, while false positives had a mean confidence of 0.8641.
- **Feature Overlap:** Misclassified fraud samples lacked typical fraudulent signatures, clustering completely within the "normal" PCA feature ranges.

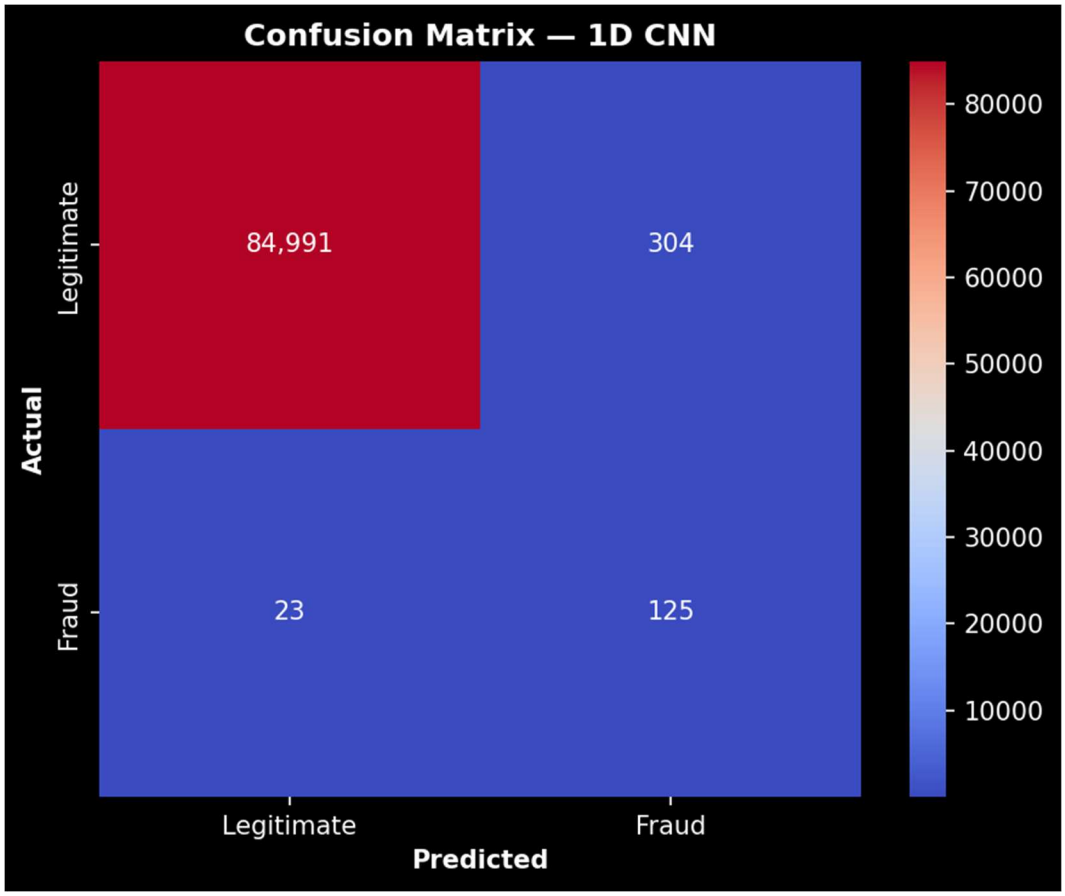

Figure 13: Error Analysis Confusion Matrix

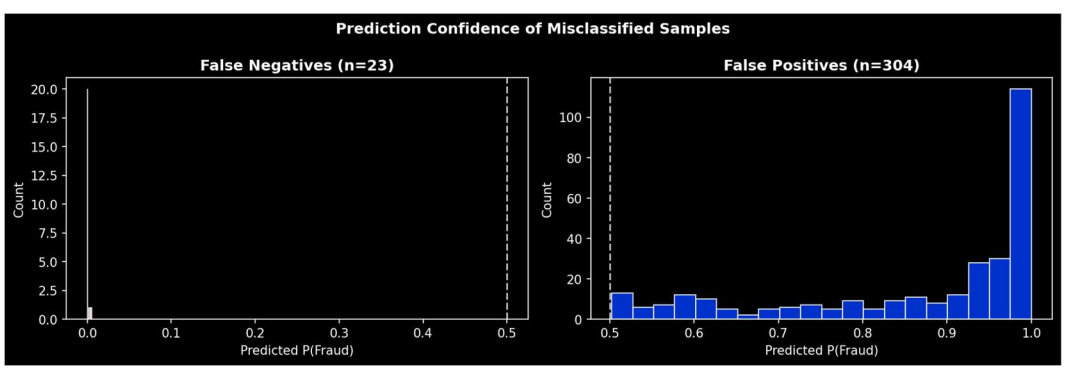

Figure 14: Confidence Distribution of Model Errors

Figure 15: Feature Distributions of Misclassified Samples

10. Individual Contributions & Team Collaboration

While the project was a collaborative effort, a parallel execution strategy was utilized for cross-verification. I independently implemented the full end-to-end pipeline in my designated directory (code_thanishk/). This dual-pipeline approach ensured that the findings were robust, reproducible, and not artifacts of a single codebase.

Collective Achievements: As a team, we collectively established the evaluation metrics, conducted the final code reviews, and synthesized the insights from our parallel pipelines to determine the optimal CNN configuration.

Individual Contributions: My individual focus (in the code_thanishk/ directory) was executing the full 1D CNN ablation study, building the MobileNetV2 foundation model pipeline, and generating the unified quantitative comparison charts.

11. Conclusion

This project successfully developed a 1D CNN for highly imbalanced tabular fraud detection. The most significant finding challenges standard deep learning assumptions: synthetic oversampling (SMOTE) degraded performance by introducing noise near the decision boundary. The model trained purely on real examples, regularized via BatchNorm, achieved a superior F1-score of 0.7930.

A key lesson learned during this project is that while synthetic oversampling (SMOTE) is a standard approach for class imbalance, it can artificially blur complex decision boundaries in tabular data.